

# Treasure Maze

Eraldi Mici

Anno accademico 2025 - 2026

## 1 Introduzione

Il progetto **Treasure Maze** ha lo scopo di risolvere due problemi di intelligenza artificiale: quello della ricerca e degli agenti e quello del machine learning attraverso l'ausilio di tecniche di computer vision.

## 2 Dominio e Vincoli

### 2.1 Dominio

La griglia è composta da  $M$  celle orizzontali e da  $N$  celle verticali, avendo così una griglia  $M \times N$ .

L'agente si trova inizialmente all'interno di una cella della griglia che contiene il carattere "S" (Start) e ha il compito di muoversi attraverso le varie celle accumulando il costo che ogni cella possiede fino ad aver raccolto il numero di tesori "T" specificato prima dell'inizio della ricerca da parte dell'agente.

La raccolta dei tesori avviene quando l'agente si trova sopra la cella e quindi il suo costo coinciderà con il costo deciso per andare nella cella contenente il tesoro.

All'interno della griglia sono presenti 4 tipi di celle:

1. Celle numerate da **0** a **4**. Queste celle contengono al loro interno un numero che rappresenta il costo per arrivare nella cella attraverso le celle adiacenti.
2. Cella **S**: Questa cella rappresenta la cella iniziale per l'agente. Se non vi è alcuna cella S la ricerca non viene iniziata.

**Assunzione:** La cella S ha costo zero per essere attraversata.

3. Cella **X**: La cella X rappresenta un muro; può essere attraversata solo dopo il suo abbattimento con un costo di 5.
4. Cella **T**: è la cella che contiene il tesoro. Il tesoro è ottenuto una volta che l'agente si trova sopra la cella. Dopo aver raccolto il tesoro.  
**Assunzione:** La cella T ha costo 1 sia prima che dopo l'ottenimento del tesoro.

## 2.2 Vincoli

L'agente è sottoposto a dei vincoli rigidi che deve seguire durante le azioni che può compiere:

1. L'agente può svolgere solo uno spostamento ad ogni mossa
2. L'agente può spostarsi solo tra celle adiacenti, ovvero: le coordinate  $x$  o  $y$  delle celle  $C_1$  e  $C_2$  hanno la seguente proprietà:  $|x_1 - x_2| + |y_1 - y_2| = 1$ .
3. È possibile attraversare una cella **X** solo dopo che è stata abbattuta.
4. Si possono raccogliere  $k$  tesori, dove  $k$  è un numero arbitrario che assume valore massimo pari al numero totale di tesori presenti nella griglia.
5. Una volta raccolto un tesoro questo non può più essere raccolto da parte dell'agente.
6. Una volta abbattuto l'ostacolo rappresentato dalla cella **X** questo assume come costo per l'attraversamento **1**.
7. Lo stato dell'agente è descritto dalla posizione in cui si trova  $x$  e  $y$  e dall'insieme di tesori raccolti e dall'insieme di muri abbattuti.
8. Lo scopo dell'agente è quello di continuare a cercare tesori finché non è stato raggiunto lo stato goal.
9. Lo stato goal consiste nella verifica del numero di tesori raccolti e il numero di tesori richiesti; è verificato nel momento in cui questi due numeri sono uguali.
10. L'agente si può muovere solo all'interno della griglia, non sono concesse mosse che lo possono portare fuori dalla griglia  $M \times N$ .

### 3 Spazio degli stati

Lo spazio degli stati di questo problema consiste nelle varie posizioni che l'agente può assumere durante la ricerca, dal possibile stato dei tesori, che può essere solo preso o ancora da prendere, dal possibile stato delle mura che può essere abbattuto o ancora da abbattere.

- #posizioni possibili dell'agente:  $N \times M$ .
- #stati possibili dei tesori:  $2^K$ , dove  $K$  è il numero totale di tesori all'interno della griglia.
- #stati possibili delle mura:  $2^H$ , dove  $H$  è il numero totale di mura all'interno della griglia.

La dimensione totale dello spazio degli stati ( $S$ ) è dunque data da:

$$S = N \cdot M \cdot 2^K \cdot 2^H$$

**Nota:** il calcolo della cardinalità degli stati possibili dei tesori e delle mura si ottiene mediante il principio fondamentale del conteggio.

### 4 Implementazione

L'implementazione dello spazio degli stati all'interno del progetto è stata fatta tramite l'utilizzo della libreria **AIMA**. Le coordinate dell'agente sono state salvate all'interno di due variabili intere `col` e `row`. Mentre le coordinate dei tesori raccolti e delle mura abbattute vengono entrambe salvate come `FrozenSet` di `Tuple`.

La griglia viene poi salvata come una lista bidimensionale dove ogni elemento della lista, è sua volta una lista e contiene una riga della griglia.

La classe che contiene lo stato dell'agente è chiamata **AgentState**.

La classe derivata da **Problem** che è quella che contiene i metodi per le **azioni** e il controllo del **goal** è chiamata **GridTreasure**.

## 4.1 Metodi

La classe `GridTreasure` possiede i metodi per trovare le prossime azioni per valutare il costo della prossima azione.

- **action**: restituisce una lista di azioni, a partire dallo stato dell'agente, che rappresentano delle stringhe delle azioni che l'agente può compiere: `left`, `right`, `up` e `down`.
- **path\_cost**: restituisce tramite `cell_cost` il costo del cammino del nodo in cui l'agente si trova verso la cella specificata dall'azione.
- **cell\_cost**: restituisce un numero che rappresenta il costo della cella specificata dall'azione. Se la cella è numerata, restituisce il numero che esso rappresenta, se la cella è un tesoro ha costo 1, e se la cella rappresenta un muro controlla se quest'ultimo fa parte dell'insieme delle mura abbattute, restituisce 1 se il muro è già stato abbattuto.
- **result**: dato lo stato attuale dell'agente e data la mossa da svolgere, restituisce un nuovo stato di `AgentState`. In questo metodo vengono raccolti i tesori e abbattute le mura.
- **find\_start**: restituisce una coppia di interi che rappresentano la colonna e riga in cui si trova lo Start.
- **goal\_test**: restituisce un valore booleano, in quanto restituisce `True` se il controllo fra il numero di tesori posseduti è uguale al numero di tesori richiesti dal problema. Altrimenti restituisce `False`.
- **treasure\_number**: restituisce un numero che rappresenta il numero totale di tesori all'interno della griglia.

## 4.2 Algoritmi di ricerca

Gli algoritmi di ricerca da parte dell'agente si possono dividere in algoritmi di ricerca informati e non informati.

### 4.2.1 Ricerca non informata

Gli algoritmi di ricerca non informati usati sono:

- **Breadth First Search:** algoritmo che esplora la griglia non tenendo conto dei costi delle singole celle, non garantendo così l'ottimalità del risultato. Ricordiamo che **BFS** garantisce la sua ottimalità quando il costo di tutte le azioni è costante.
- **Depth First Search:** algoritmo di ricerca in profondità che risulta essere il più veloce in quanto estrae il primo cammino che soddisfa i requisiti della ricerca non tenendo conto però dell'ottimalità del risultato. **DFS** ha quindi un costo nello spazio utilizzato pari a:  $O(bm)$  dove  $b$  è il fattore di branching e  $m$  è la lunghezza del percorso effettuato dall'agente prima di fare backtracking.
- **Uniform Cost Search:** algoritmo di ricerca che espande i nodi in ordine crescente di costo cumulato dal nodo iniziale al nodo corrente. In questo modo vengono considerati per primi i cammini meno costosi e, in presenza di costi non negativi, l'algoritmo garantisce l'ottimalità della soluzione.

#### 4.2.2 Ricerca informata

Gli algoritmi di ricerca informata prevedono l'utilizzo di un'**euristica**, ovvero una funzione che nel nostro caso stima quanto siamo vicini al tesoro più vicino. Come algoritmo di ricerca informato usiamo **A\*** che è definita come:  $f(n) = g(n) + h(n)$ . Come  $g(n)$  usiamo il costo cumulato del cammino dal nodo iniziale al nodo corrente, stesso criterio usato da UCS, mentre come euristica  $h(n)$  sono state usate 2 varianti:

- **Manhattan:** questa euristica stima la distanza dal goal in questo modo  $d_{Manhattan}(P_1, P_2) = |x_1 - x_2| + |y_1 - y_2|$ . In pratica misuriamo quanto è distante il goal in termini di passi che l'agente può compiere.
- **Čebyšëv:** stima la distanza dal goal come:  $d_{Čebyšëv}(P_1, P_2) = \max(|x_1 - x_2|, |y_1 - y_2|)$ . Con Čebyšëv si stima la distanza dal goal considerando il massimo tra gli scarti assoluti sulle coordinate. Normalmente questa metrica viene usata in casi in cui è concesso il movimento diagonale, cosa che nel nostro caso non è concesso, quindi risulterà essere più sottostimante di Manhattan.

Si ricordi che per essere ottimale l'algoritmo **A\*** richiede che le euristiche siano ammissibili, ovvero che le stime fatte non devono essere maggiori della

distanza reale dal nodo corrente al goal. Nel nostro caso sia **Manhattan** che **Čebyšëv** sottostimano al più la distanza dal goal:  $0 \leq h(n) \leq h^*(n)$ , dove  $h^*(n)$  è la vera distanza dal nodo al goal.

L'ammissibilità di Manhattan è garantita dal fatto che la stima che fa può essere solo minore del costo effettivo in quanto la presenza di ostacoli non può che far sottostimare il costo effettivo. Mentre l'ammissibilità di Čebyšëv è garantita dal fatto che la formula stessa non può essere solo che minore o uguale della stima di Manhattan, quindi questo implica il fatto che A\* fornita di questa euristica tenderà anche ad espandere più nodi di A\* usando l'euristica Manhattan.

## 5 Modello di classificazione

Per la classificazione dei caratteri che ogni cella della griglia contiene è stato usato un modello di *machine learning* di tipo **Multi-Layer perceptron**. La rete è una rete **feed-forward** densa, ovvero, ogni nodo presenta una connessione con ogni nodo nel layer successivo. La rete presenta:

- **Layer di input:** presenta una dimensione pari alle feature significative per il problema, essendo un problema di classificazione legato alla natura dell'immagine, ogni pixel risulta essere significativo. il numero di pixel dell'immagine è pari a:  $28 * 28 = 784$  pixel.
- **Primo layer nascosto:** Per il primo layer nascosto sono stati scelti 256 nodi, come convenzione è stata usata una potenza di 2; empiricamente è stato notato un notevole miglioramento da parte del modello a riconoscere i caratteri con il primo layer nascosto non minore del valore scelto.
- **Secondo layer nascosto:** per il secondo layer sono stati scelti 128 nodi. La scelta del numero di neuroni è stata fatta per favorire una maggiore astrazione, da parte del modello, di caratteristiche sintetiche e discriminanti.
- **Layer di output:** per il layer di output la scelta del numero di neuroni deve coincidere con il numero di label che il modello deve classificare, ovvero nel nostro caso 8 neuroni.

Mentre le funzioni di attivazione usate sono:

- **ReLU** per il primo e il secondo layer. Questa funzione è spesso utilizzata nei layer nascosti per aggiungere non linearità al modello.

$$f(x) = \max(0, x)$$

- **Softmax** per il layer di output. L'utilizzo di softmax per il layer di output è tipico dei problemi di classificazione dove deve essere scelta un'etichetta da dare all'immagine, in quanto assegna ad ogni neurone di output una probabilità corrispondente all'etichetta.

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

L'utilizzo di funzioni di attivazione è fondamentale per l'introduzione di non linearità, permette al modello di apprendere pattern più complessi.

L'ottimizzatore scelto è **Adam**; impiegato per aggiornare i pesi della rete durante la fase di apprendimento. È una versione migliorata del metodo della discesa del gradiente, che risolve alcune delle sue problematiche. Aiuta a mitigare alcune criticità di quest'ultima, come la sensibilità alla scelta del learning rate, la presenza di oscillazioni durante l'ottimizzazione, la lentezza della convergenza e le difficoltà in presenza di gradienti sparsi.

Come funzione di loss, ovvero la funzione che decide quanto è grande l'errore commesso dal modello nella predizione e che viene propagato all'indietro, è stata usata **sparse categorical cross entropy**, una funzione che lavora con label discrete.

$$L = -\log(\hat{y}_{true})$$

Per prevenire l'overfitting è stato scelto un metodo **early stopping** che monitora il **valore di loss** considerato sul validation set e determina, se questo valore smette di diminuire, se fermare il training e portarlo nell'epoca in cui non causava più *overfitting*. Come valore di **patience** è stato scelto il valore di 3, ovvero si aspettano 3 epoche in cui il modello smette di migliorare.

Per il modello è stato scelto un validation split del 10% che è stato ricavato dai dati per il training.

Come dimensione del batch, ovvero dimensione del gruppo di sample che vengono dati al modello prima che avvenga l'aggiornamento dei pesi è stata scelta una dimensione di 128.

## 6 Dataset

Il **Dataset** scelto per l'addestramento del modello è **EMNIST**, ovvero un dataset contenente immagini di lettere minuscole e maiuscole dell'alfabeto inglese, comprendente le 26 lettere dell'alfabeto più i 10 caratteri dei numeri arabi, tutte scritte a mano.

Il dataset è stato filtrato, in quanto, come riportato nel dominio del problema, il modello deve solo riconoscere lettere maiuscole e, per di più, è stato scelto solo un suo sottoinsieme, ovvero solo le lettere "S", "T" e "X"; la stessa cosa vale per le cifre; difatti, sono state prese per il training e per il test solo le cifre da 0 a 4.

Le dimensioni per il set di training e per il test sono rispettivamente:

- **19200** immagini per il set di training.
- **3200** immagini per il set di test.

Al dataset è stato inoltre fatto un lavoro di preprocessing in quanto le immagini di default sono specchiate e sono state ruotate di 90 gradi. Dopo di che i dati contenuti dall'immagine (i pixel) sono stati normalizzati, ovvero ogni pixel ora è compreso fra 0 e 1, in modo tale da rendere l'allenamento più efficiente tramite la discesa del gradiente.

## 7 Estrazione della griglia

Per estrarre le celle dalla griglia è stata usata la libreria **OpenCV**, essendo il programma pensato anche per funzionare in linea di principio anche con griglie disegnate a mano, l'estrazione delle celle deve seguire regole quanto più flessibili; l'estrazione delle celle segue la seguente pipeline:

1. L'immagine viene letta e convertita in scala di grigi tramite.



2. Applichiamo del padding e un threshold adattivo all'immagine, questo per riuscire a lavorare meglio con immagini più generiche possibili e con condizioni di luce differente; infine applichiamo un effetto di blur per ridurre il rumore dell'immagine.
3. Dopo la pulizia dell'immagine estraiamo le principali linee verticali ed orizzontali che formano la griglia.
4. Dopo aver estratto le linee verticali ed orizzontali, determiniamo i poligoni che si vengono a formare.
5. Dopo aver determinato le possibili linee che formano la griglia, filtriamo solo quelle che ci interessano.
6. A partire delle linee estratte, troviamo le celle che soddisfano i requisiti che classificano la figura come cella.
7. Ordiniamo e ripuliamo le celle trovate in modo tale poterle valutare e salvarle in una lista bidimensionale da utilizzare in seguito.

## 8 Risultati

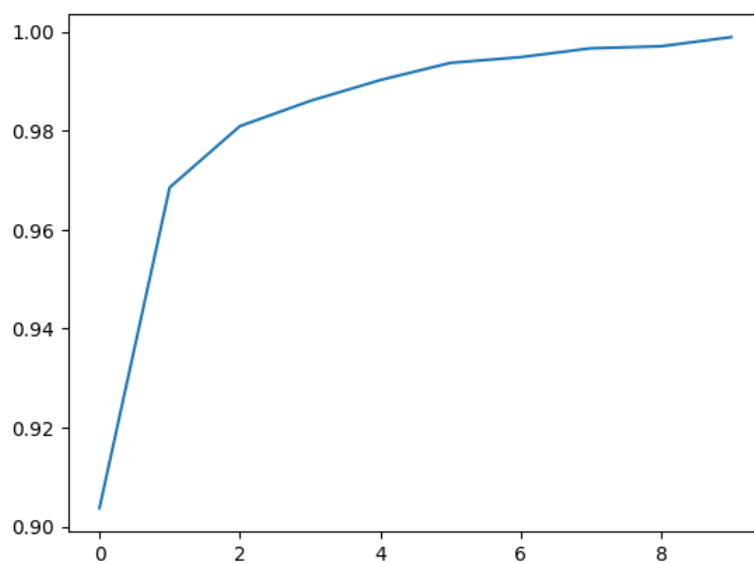
### 8.1 Performance del modello a seguito del training

Il modello a seguito del training, usando una *patience* di 3 epoche, presenta, alla fine del training, le seguenti statistiche:

**Nota:** i dati in questione sono specifici di un singolo training ma si attestano essere come rappresentativi di un training medio del modello.

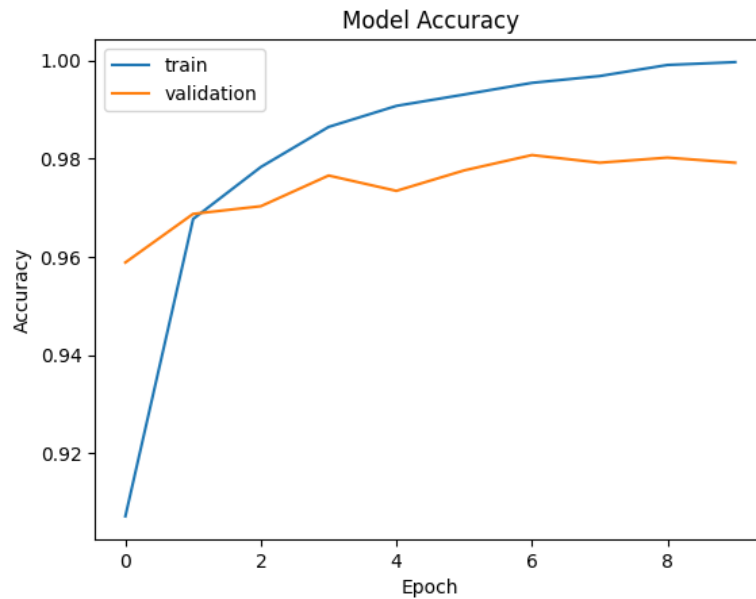
- **accuracy:** 0.9989 - la percentuale delle predizioni corrette del modello.
  - **val. accuracy:** 0.9839 - percentuale delle predizioni corrette questa volta fatta sul validation set.
- **loss:** 0.0060 - valore che indica l'errore commesso dal modello.
  - **val. loss:** 0.0661 - valore dell'errore commesso sul validation set.

Questa è la performance dell'accuracy del modello nel training set, nel corso delle epoche:



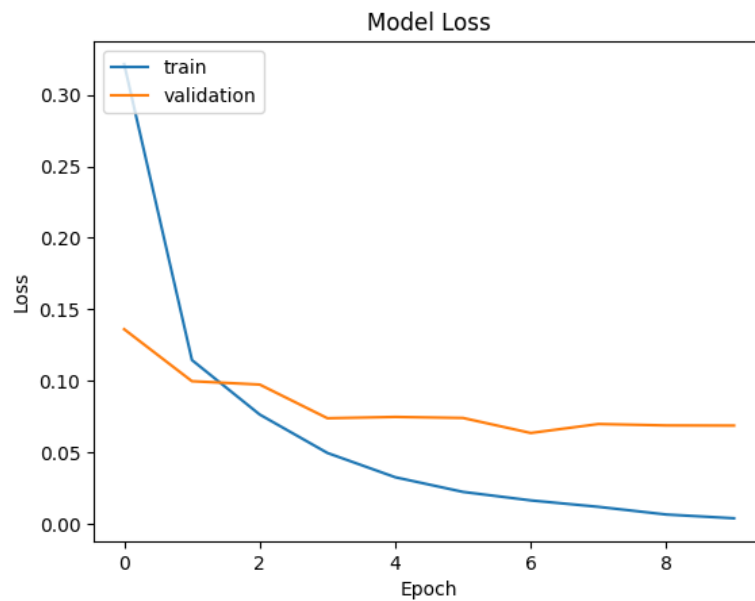
Come si può notare dalla crescita del grafico, il modello migliora nella qualità delle sue predizioni.

Mentre per quanto riguarda l'accuratezza del modello rispettivamente nel training set e nel validation set, abbiamo il seguente grafico:

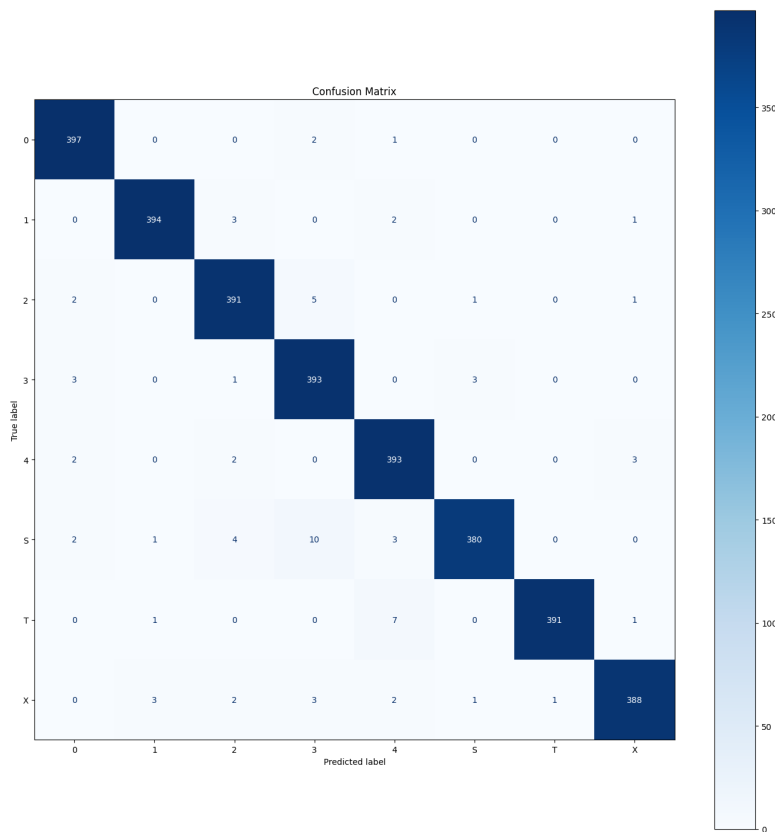


Il grafico suggerisce una performance complessivamente buona del modello, in quanto entrambe le accuracy del modello si attestano con valori alti; seppure il comportamento nel validation set potrebbe suggerire un leggero overfitting nelle ultime epoche, suggerito da un suo appiattimento e dal divario crescente ottenuto tra il training e il validation set.

Lo stesso possiamo dire del grafico della funzione di loss. Confrontando l'andamento delle funzioni di training e del validation set, appuriamo un leggero appiattimento della funzione di loss per quanto riguarda il validation set oltre che una differenza di valori assunti con il passare delle epoche; ma comunque il modello mantiene delle ottime performance.



La **confusion matrix**, ossia la matrice di confusione, mostra la distribuzione delle predizioni del modello sul test set confrontando le classi reali con quelle predette. Nel caso in esame, la prevalenza di valori lungo la diagonale principale evidenzia prestazioni complessivamente molto buone.



Come è possibile vedere, la maggior parte delle predizioni fatte sono sulla diagonale, ovvero le classi predette sono quelle reali e quindi il modello si è comportato bene.

Tra tutte le classi quella che fa più fatica ad essere riconosciuta è la classe S, difatti su 400 esempi posti al modello, viene predetta correttamente 380 volte. Sulle 20 predizioni non corrette viene predetta come appartenente alla classe **3** dieci volte.

Questo è un altro segnale delle ottime performance del modello.

Provando ad analizzare altre metriche che ci possono suggerire comportamenti indesiderati del modello arriviamo allo studio di: precision, recall, f1-score e support.

| Classe       | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.98      | 0.99   | 0.99     | 400     |
| 1            | 0.99      | 0.98   | 0.99     | 400     |
| 2            | 0.97      | 0.98   | 0.97     | 400     |
| 3            | 0.95      | 0.98   | 0.97     | 400     |
| 4            | 0.96      | 0.98   | 0.97     | 400     |
| S            | 0.99      | 0.95   | 0.97     | 400     |
| T            | 1.00      | 0.98   | 0.99     | 400     |
| X            | 0.98      | 0.97   | 0.98     | 400     |
| Accuracy     |           | 0.98   |          | 3200    |
| Macro avg    | 0.98      | 0.98   | 0.98     | 3200    |
| Weighted avg | 0.98      | 0.98   | 0.98     | 3200    |

Le performance mostrano ancora una volta, come il modello riesca a generalizzare bene il problema di classificare le immagini. La **precision** media di tutte le classi è molto alta; il modello predice con abbastanza precisione tutte le classi. Mentre la **recall** si attesta ad un valore molto alto di 0.98, ovvero riesce quasi sempre a individuare esempi reali appartenenti alle diverse classi; difatti il valore di F1-score è in linea con quanto riportato fino ad ora.

## 9 Uso del modello con esempi presi fuori dal Dataset

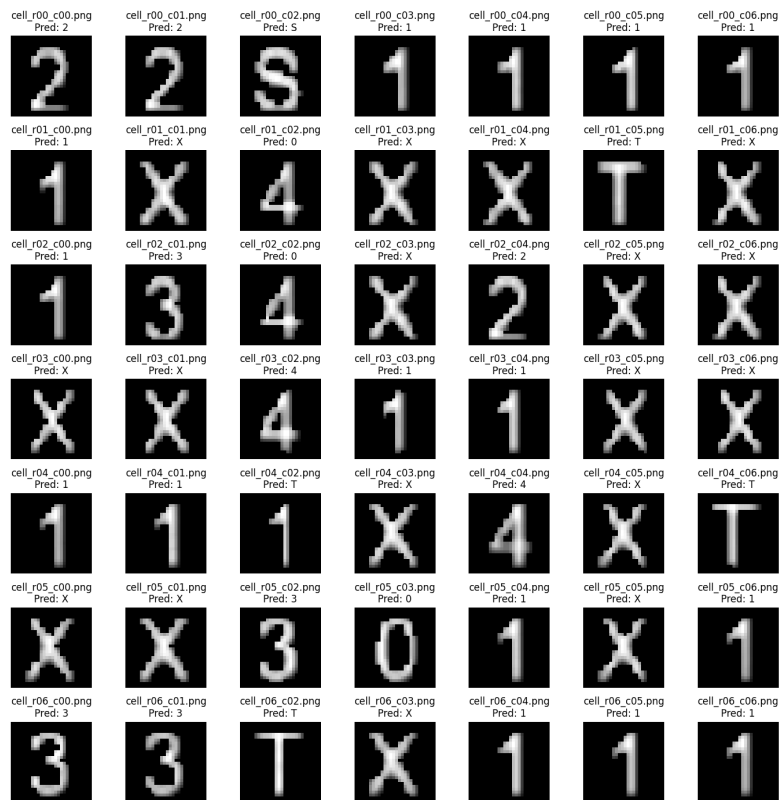
Per testare il modello dopo il suo training, sono state usate griglie create digitalmente e create a mano, quindi sia caratteri dello stesso genere del dataset (caratteri scritti a mano) che caratteri fatti al computer.

Per il primo test della griglia fatta a mano è stata usata la seguente immagine:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 2 | S | 1 | 1 | 1 | 1 |
| 1 | X | 4 | X | X | T | X |
| 1 | 3 | 4 | X | 2 | X | X |
| X | X | 4 | 1 | 1 | X | X |
| 1 | 1 | 1 | X | 4 | X | T |
| X | X | 3 | 0 | 1 | X | 1 |
| 3 | 3 | T | X | 1 | 1 | 1 |

Come è possibile vedere è una griglia quadrata 7x7, che presenta lo Start rispettivamente alla riga e alla colonna: (1,3); sono presenti in totale 3 tesori. La prima cosa che succede è l'estrazione delle celle dalla griglia, vengono estratte correttamente tutte e 49 le celle e salvata in una cartella. Dopo di che ogni cella viene sottoposta ad una procedura di preprocessing e valutata dal modello.

Il risultato della valutazione del modello è il seguente:



Come è possibile notare il modello riesce a predire 46 celle su 49. Gli errori commessi sono su due celle contenenti il numero 4 scambiati per uno 0 e per un 1 scambiato con una T. L'esecuzione dell'agente, eseguito per prendere tutti i tesori, produce i seguenti risultati:

**Nota:** il tempo di esecuzione dipende dalla macchina che esegue il programma, è stato riportato questo valore per mostrare la differenza di ordine di grandezza tra i vari algoritmi e quale fra questi viene eseguito prima.

- **Depth-First-Search**
  - Costo del cammino: 135
  - Lunghezza del cammino: 88
  - Tempo impiegato: 0.0041s
- **Uniform-Cost-Search**



- Nodi generati: 23308
- Nodi esplorati: 6751
- Costo del cammino: 23
- Lunghezza del cammino: 34
- Tempo impiegato: 24.2606s
- **A\*** con euristica Manhattan
  - Nodi generati: 14056
  - Nodi esplorati: 4005
  - Costo del cammino: 23
  - Lunghezza del cammino: 34
  - Tempo impiegato: 11.7344s
- **A\*** con euristica Čebyšëv
  - Nodi generati: 16529
  - Nodi esplorati: 4711
  - Costo del cammino: 23
  - Lunghezza del cammino: 34
  - Tempo impiegato: 14.8671s

Come appurato nello studio del **Capitolo 4.2.2**, l'algoritmo migliore nella pratica, nel test riportato, fra quelli utilizzati nella ricerca informata è **A\* con l'euristica di Manhattan** a parità di ottimalità ha esplorato e generato meno nodi di tutti gli altri algoritmi, impiegandoci così meno tempo fra quelli ottimali. Poco sotto abbiamo **A\* con l'euristica di Čebyšëv**; genera ed esplora leggermente più nodi della sua variante Manhattan e il suo tempo di esecuzione è per questo leggermente maggiore. Mentre **Uniform-Cost-Search** risulta essere meno efficiente di Manhattan e di Čebyšëv in termini di tempo, in quanto non presenta la componente euristica che tende ad indirizzare l'agente verso il tesoro più vicino ma ha comunque garantito l'ottimalità. Infine l'algoritmo più veloce risulta essere **DFS** ma non ha garantito l'ottimalità della soluzione avendo mostrato il cammino più costoso fra tutti gli algoritmi.

## 10 Limiti

Durante l'utilizzo del programma sono state notate alcune limitazioni:

1. Non sempre l'estrazione della griglia avviene correttamente; in alcuni casi vengono considerate celle che in realtà non contengono alcuna immagine rilevante per il problema.
2. Dal punto 1. ne consegue che la griglia che si viene a formare non è corretta e quindi il problema che si risolve non è esattamente quello richiesto.
3. Se sono presenti due Start (poiché un'immagine è stata erroneamente classificata come S) i tempi di esecuzione aumentano.
4. Se per qualche motivo sono presenti due start viene considerato valido il primo che viene trovato seguendo l'ordine di scansione della griglia (dall'alto verso il basso e da sinistra verso destra).
5. Il tempo di esecuzione del programma cresce al crescere della griglia.
6. Se non viene riconosciuto lo Start, l'agente non viene eseguito.

Il limite più grande del programma è sicuramente la parte di computer vision che compie la maggior parte degli errori che di conseguenza non fa compiere una valutazione appropriata al modello.